Estructuras de Datos

2026
Primer Cuatrimestre

REPASO

Árboles Binarios

Un árbol binario es un árbol ordenado que cumple:

1) Cada nodo tiene a lo sumo dos hijos (0, 1 o 2)

2) Cada nodo hijo es o bien hijo izquierdo o hijo derecho

3) El hijo izquierdo precede al hijo derecho en el orden de los hijos de un nodo

Árboles Binarios

El subárbol que tiene como raíz al hijo izquierdo se llama subárbol izquierdo.

El subárbol que tiene como raíz al hijo derecho se llama subárbol derecho.

En un árbol binario propio, cada nodo tiene 0 o dos hijos (el libro también lo llama *full BT*). Si un árbol binario no es propio, entonces es impropio.

ÁRBOL BINARIO DE BÚSQUEDA

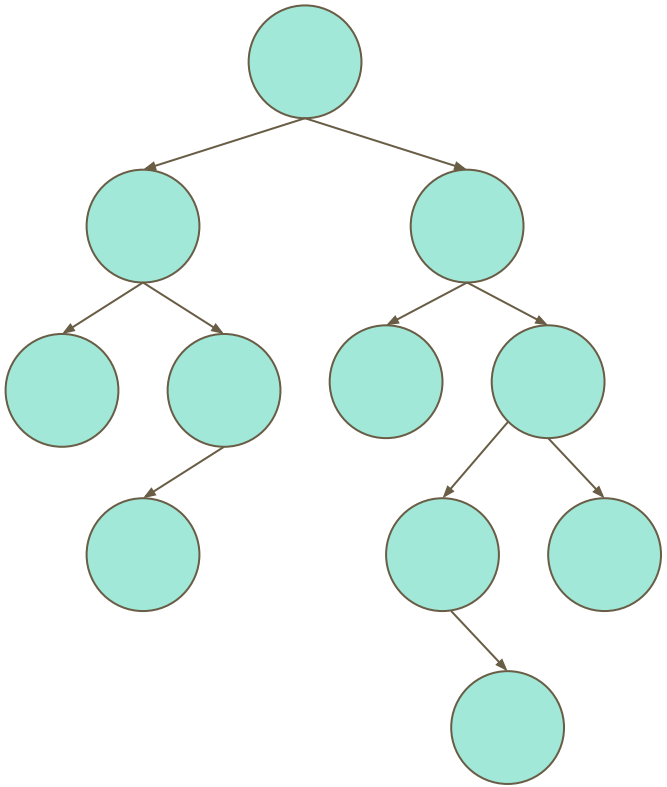
Árbol Binario de Búsqueda

- El árbol binario de búsqueda (ABB) es una **estructura de datos** útil para implementar conjuntos, mapeos y diccionarios.
- En un ABB las claves en los nodos se hallan ordenadas de una manera particular.
- El tiempo de insertar, recuperar y eliminar es proporcional a la altura del ABB.
    - Si el árbol tiene n claves, la altura del ABB se halla entre $\log_2(n)$ y n.

Árbol Binario de Búsqueda

Un ABB es un árbol binario tal que:

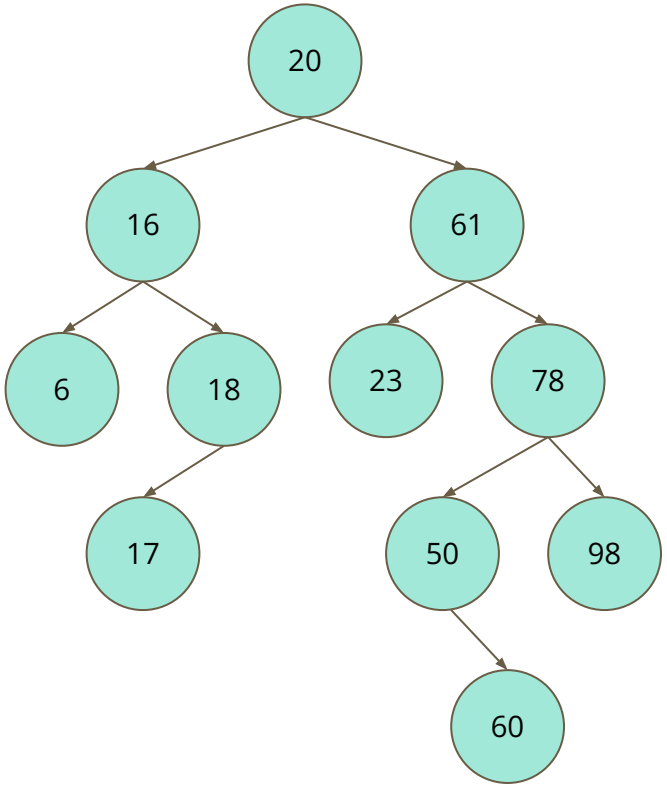
- Es vacío, o
- Es un nodo con rótulo k e hijos t_{izq} y t_{der} tales:
  - k > claves de t_{izq},
  - k < claves de t_{der}, y,
  - t_{izq} y t_{der} son ABB.



Árbol Binario de Búsqueda

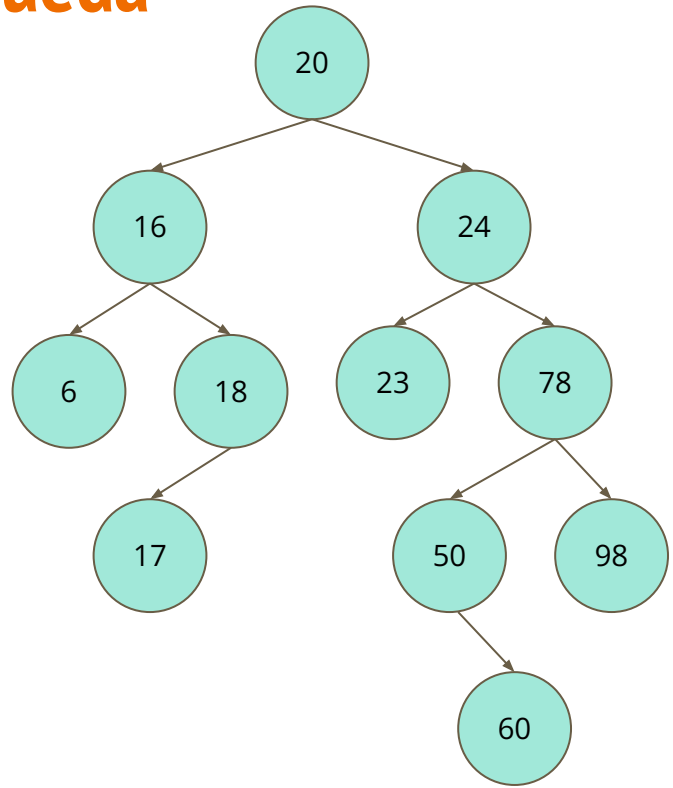
Un ABB es un árbol binario tal que:

- Es vacío, o
- Es un nodo con rótulo k e hijos t_{izq} y t_{tder} tales:
  - k > claves de t_{izq},
  - k < claves de t_{der}, y,
  - t_{izq} y t_{der} son ABB.



Árbol Binario de Búsqueda - Búsqueda

La búsqueda usa el ordenamiento de los rótulos para recorrer el árbol.



Árbol Binario de Búsqueda - Búsqueda

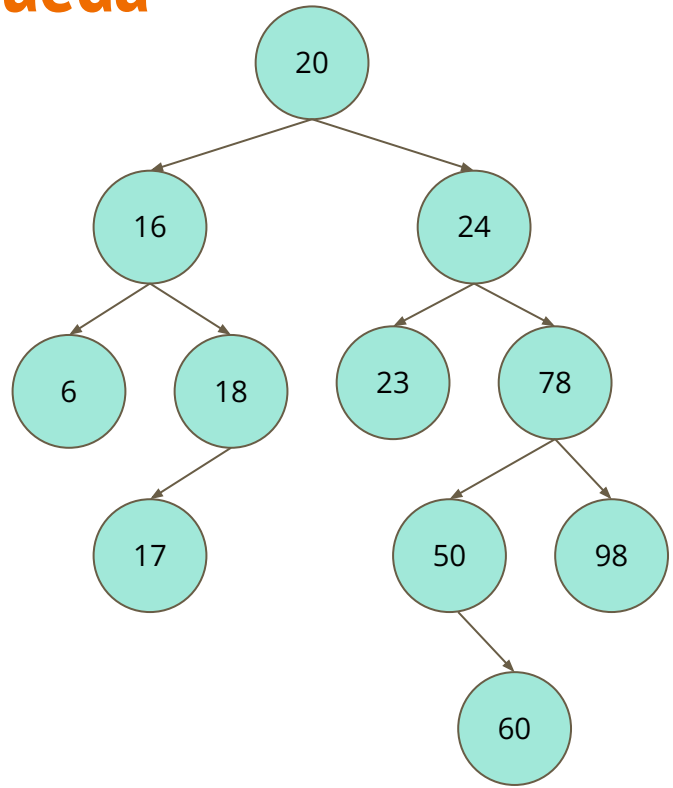
Buscar(x, p):

CB: Si estoy en un nodo vacío p =>
x no está en el árbol

CB: Si estoy en un nodo no vacío p y x=rótulo(p) =>
x está en el árbol

CR: Si estoy en un nodo no vacío p y x<rótulo(p) =>
Buscar(x,p) = Buscar(x, hijo izquierdo de p)

CR: Si estoy en un nodo no vacío p y x>rótulo(p) =>
Buscar(x,p) = Buscar(x, hijo derecho de p)



Árbol Binario de Búsqueda - Búsqueda

Buscar(x, p):

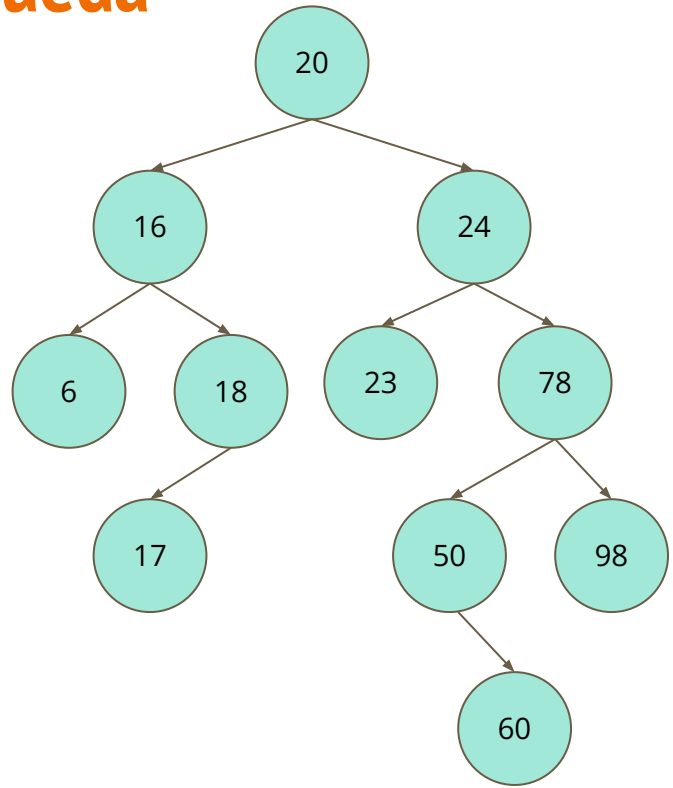
CB: Si estoy en un nodo vacío p =>
x no está en el árbol

CB: Si estoy en un nodo no vacío p y x=rótulo(p) =>
x está en el árbol

CR: Si estoy en un nodo no vacío p y x<rótulo(p) =>
Buscar(x,p) = Buscar(x, hijo izquierdo de p)

CR: Si estoy en un nodo no vacío p y x>rótulo(p) =>
Buscar(x,p) = Buscar(x, hijo derecho de p)

Busquemos a 50



Árbol Binario de Búsqueda - Búsqueda

Buscar(x, p):

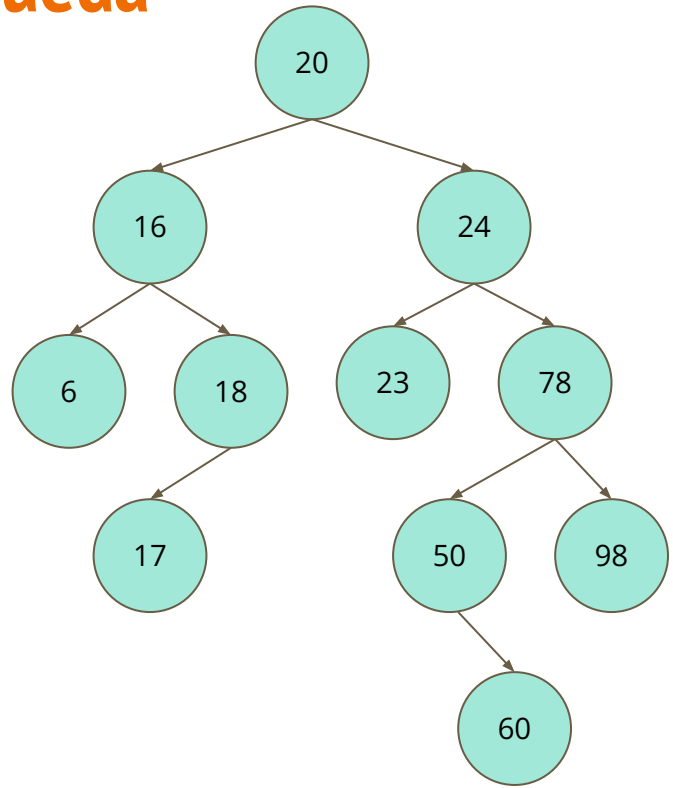
CB: Si estoy en un nodo vacío p =>
x no está en el árbol

CB: Si estoy en un nodo no vacío p y x=rótulo(p) =>
x está en el árbol

CR: Si estoy en un nodo no vacío p y x<rótulo(p) =>
Buscar(x,p) = Buscar(x, hijo izquierdo de p)

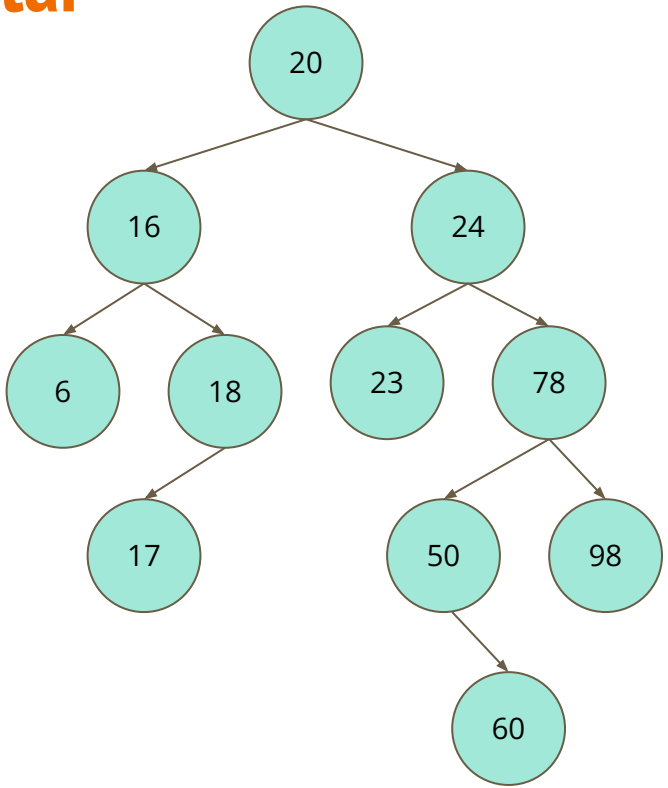
CR: Si estoy en un nodo no vacío p y x>rótulo(p) =>
Buscar(x,p) = Buscar(x, hijo derecho de p)

Busquemos a 19



Árbol Binario de Búsqueda - Insertar

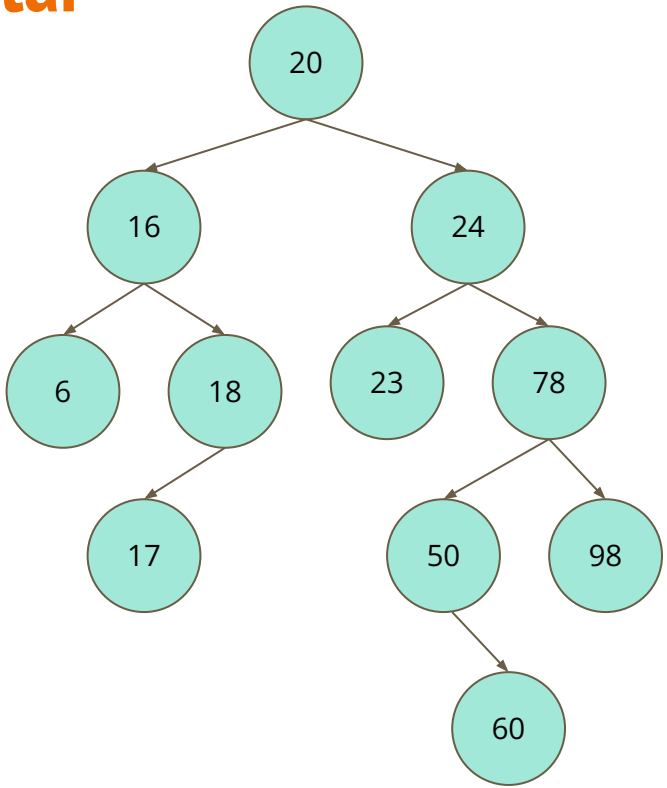
Las nuevas claves siempre se insertan como un nodo hoja.



Árbol Binario de Búsqueda - Insertar

Algoritmo insertar(k, p) { Comienza en la raíz del ABB }

```
Si p es vacío entonces
        crear un nodo hoja con rótulo k
Sino
        si k < clave(p)
                entonces
                        insertar(k, hijo izquierdo de p)
                sino
                        si k > clave(p) entonces
                                insertar( k, hijo derecho de p )
                        sino
                                si k = clave(p) entonces
                                        reemplazar rótulo de p
```

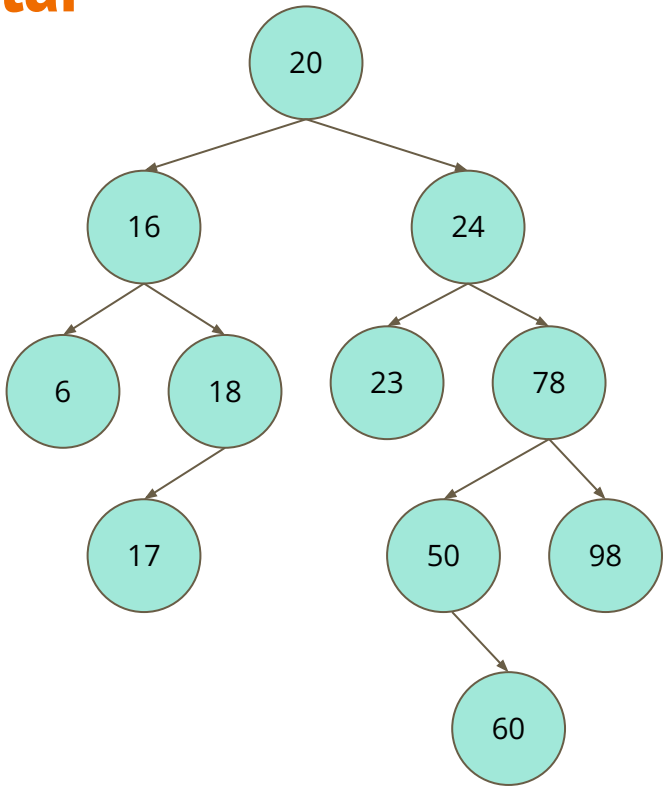


Árbol Binario de Búsqueda - Insertar

Algoritmo insertar(k, p) { Comienza en la raíz del ABB }

```
Si p es vacío entonces
        crear un nodo hoja con rótulo k
Sino
        si k < clave(p)
                entonces
                        insertar(k, hijo izquierdo de p)
                sino
                        si k > clave(p) entonces
                                insertar( k, hijo derecho de p )
                        sino
                                si k = clave(p) entonces
                                        reemplazar rótulo de p
```

Nota: La operación "*reemplazar rótulo de p*" dependerá de si:
(a) Implemento un conjunto: no hacer nada
(b) Implemento un mapeo: modifico el valor de la entrada
(c) Implemento un diccionario: ¿?

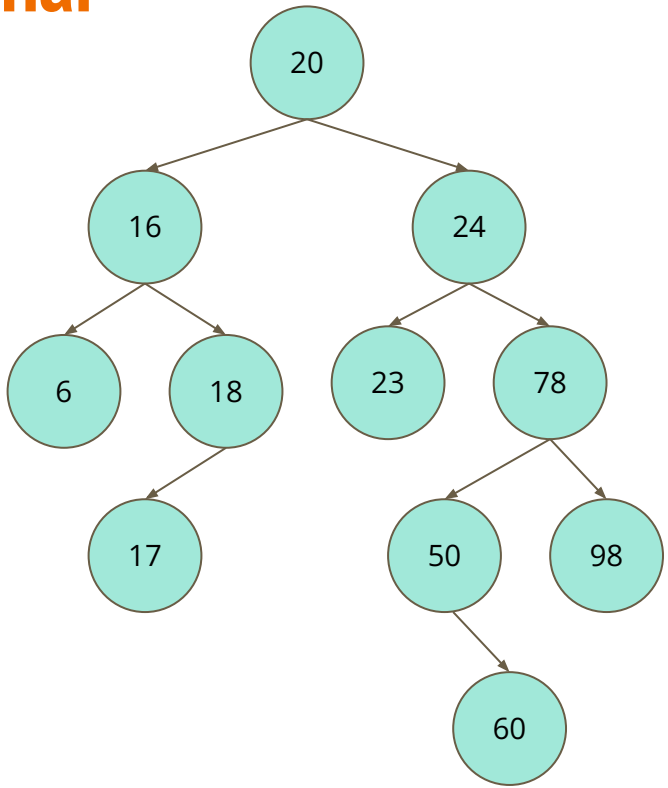


Árbol Binario de Búsqueda - Eliminar

Dos estrategias:

1) **Eliminación perezosa (lazy delete):**

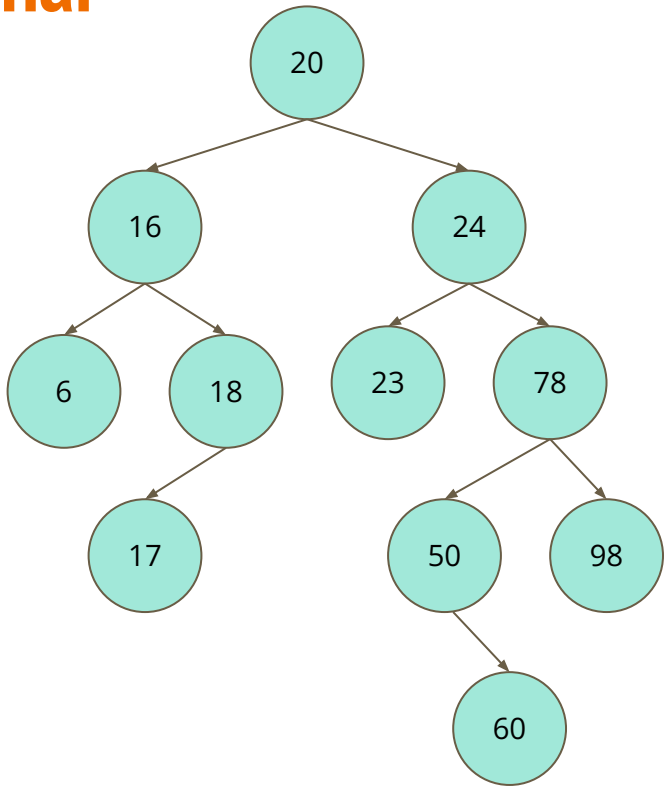
- Busco el dato a eliminar y marco el nodo como eliminado (sin sacarlo del árbol).
- Se necesita un campo más en el NodoABB de tipo boolean que vale true si el dato ha sido eliminado.
- Member e Insert deben ser modificados para tener en cuenta esto;
  - Cuando member encuentra el dato en el árbol debe además ver si el dato no está eliminado.
  - Cuando insert encuentra el dato debe ver si no es necesario setear eliminado en falso.



Árbol Binario de Búsqueda - Eliminar

Dos estratégias:

2) **Eliminación física:** Busco el dato y elimino el nodo del árbol.



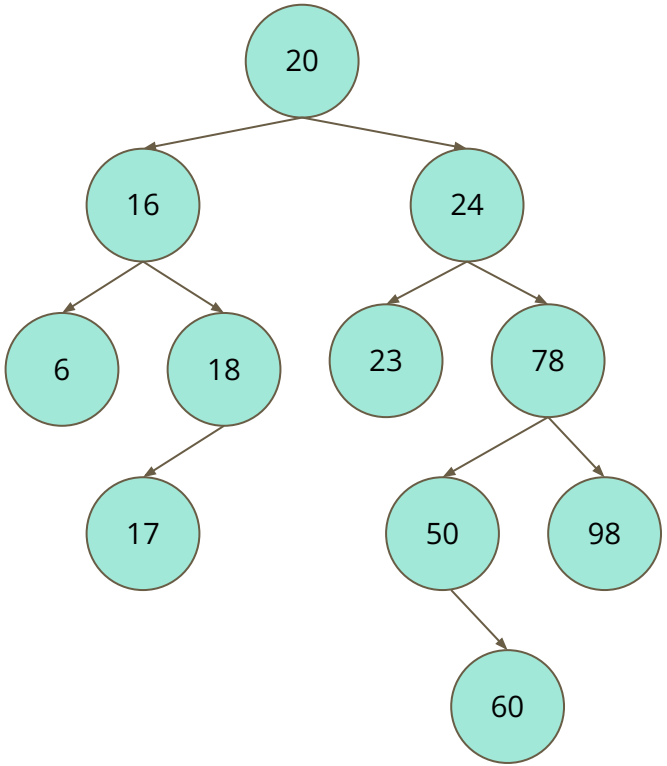
Árbol Binario de Búsqueda - Eliminación Física

Eliminar (x)

Paso 1: Buscar el nodo P con la clave x a eliminar

Paso 2: Cuatro alternativas

a) P es una hoja: Eliminarlo
b) P tiene sólo un hijo izquierdo: Bypass del padre de p al hijo izq de p
c) P tiene sólo un hijo derecho: Bypass del padre de p al hijo der de p
d) P tiene dos hijos: Reemplazar el rótulo de p con el sucesor inorder y eliminar el sucesor inorder de p.



Árbol Binario de Búsqueda - Eliminación Física

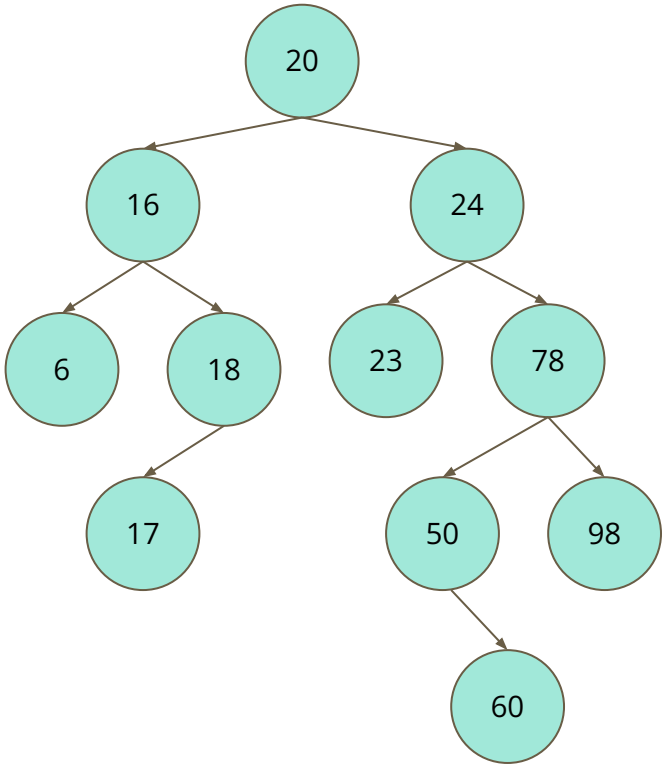
Eliminar (x)

Paso 1: Buscar el nodo P con la clave x a eliminar

Paso 2: Cuatro alternativas

P tiene dos hijos: Reemplazar el rótulo de p con el sucesor inorder y eliminar el sucesor inorder de p.

El sucesor inorder de p se encuentra caminando un paso hacia la derecha y luego hacia la izquierda todo lo que sea posible hasta llegar a un nodo sin hijo izquierdo. El nodo encontrado es una hoja o solo tiene hijo derecho. Eliminarlo cae en los casos (1) ó (2).



Análisis de la Complejidad Temporal

Sea h la altura del ABB y sea n la cantidad de claves del ABB

$$T(h) = \begin{cases} c_1 & si \quad h = 0 \\ c_2 + T(h-1) & si \quad h > 0 \end{cases}$$

con lo cual *T(h) = O(h)*

Análisis de la Complejidad Temporal

$T(h) = O(h)$

La altura en el peor caso es $h = n - 1 = O(n)$ (se da cuando se hicieron inserciones de claves en forma ascendente o descendente).

La altura en el mejor caso es $h = O(\log_2(n))$ ya que las inserciones producen un árbol lleno que tiene $n = 2^{h+1-1}$ nodos; de ahí, despejo $h = log_2(n + 1) - 1$.

El árbol lleno de altura h es aquel que todos los nodos internos tienen 2 hijos y todas las hojas tienen la misma profundidad h.

Implementación de ABB

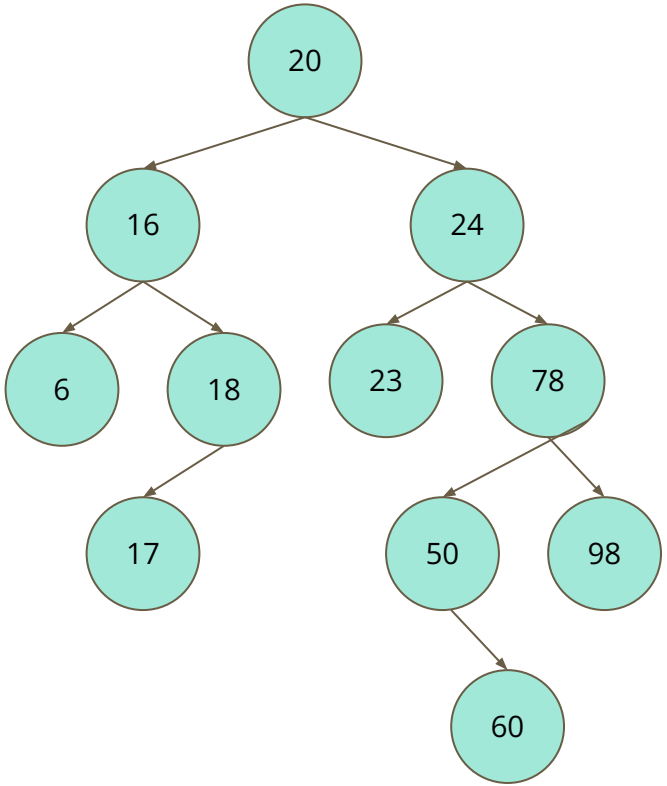
Para facilitar la programación usaremos un árbol propio donde cada hoja tiene dos hijos dummy (placeholder según GT).

Estos dummies sirven para insertar nuevas claves en su lugar y representan ABB vacíos.

Implementación de ABB

Para facilitar la programación usaremos un árbol propio donde cada hoja tiene dos hijos dummy (placeholder según GT).

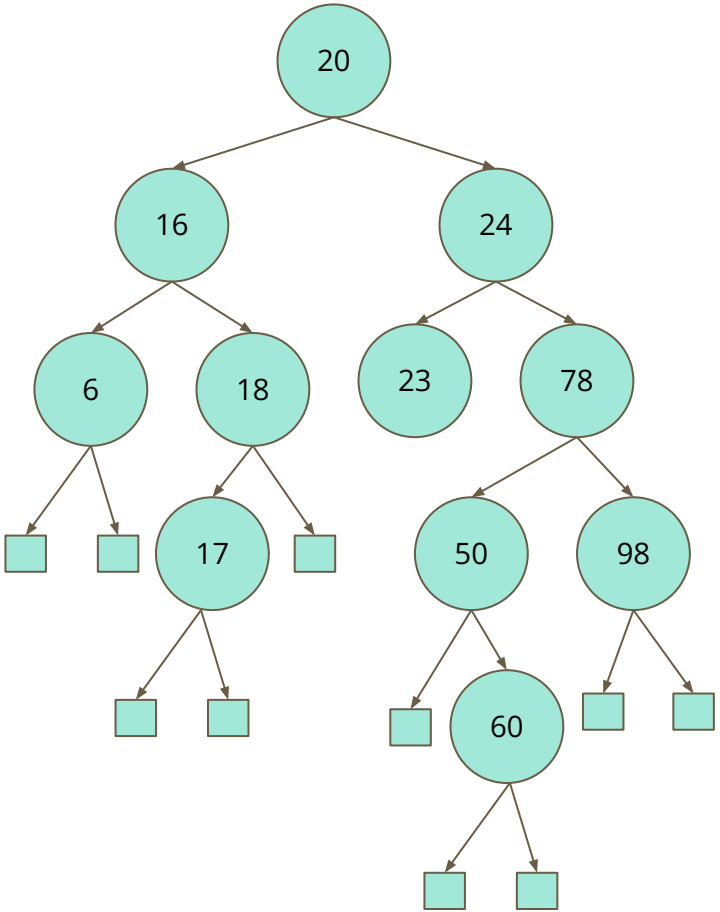
Estos dummies sirven para insertar nuevas claves en su lugar y representan ABB vacíos.



Implementación de ABB

Para facilitar la programación usaremos un árbol propio donde cada hoja tiene dos hijos dummy (placeholder según GT).

Estos dummies sirven para insertar nuevas claves en su lugar y representan ABB vacíos.



Implementación de ABB

```
import java.util.Comparator;
        public class ABB<E> {
                protected NodoABB<E> raiz;
                protected Comparator<E> comp;
                public ABB(Comparator<E> comp) {
                        // Creo un nodo dummy como raíz del árbol
                        // para dejarlo preparado para la primera inserción:
                        raiz = new NodoABB<E>(null,null);
                        this.comp = comp; }
```

Implementación de ABB

```
public NodoABB<E> buscar( E x ) {
        return buscarAux( x, raiz ); // Búsqueda recursiva desde la raíz }
private NodoABB<E> buscarAux( E x, NodoABB<E> p ) {
        if( p.getRotulo() == null ) return p; // Llegué a un dummy
        else { // Estoy en un nodo con un dato no nulo
                int c = comp.compare( x, p.getRotulo() );
                if( c == 0 ) return p; // Lo encontré porque x = p.getRotulo()
                else if( c < 0 ) /* Busco a izq porque x < p.getRotulo() */
                        return buscarAux( x, p.getIzq() );
                else /* Busco a derecha porque x > p.getRotulo() */
                        return buscarAux( x, p.getDer() ); } }
```

Implementación de ABB

```
public class NodoABB<E> {
        private E element;
        private NodoABB<E> left, right, parent;

        public NodoABB(E e, NodoABB<E> p){
                element=e;
                setParent(p);
                left=setLeft(null);
                right=setRight(null);     }
```

Bibliografía

Capítulo 10 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.